

ea源码使用文档

环境准备

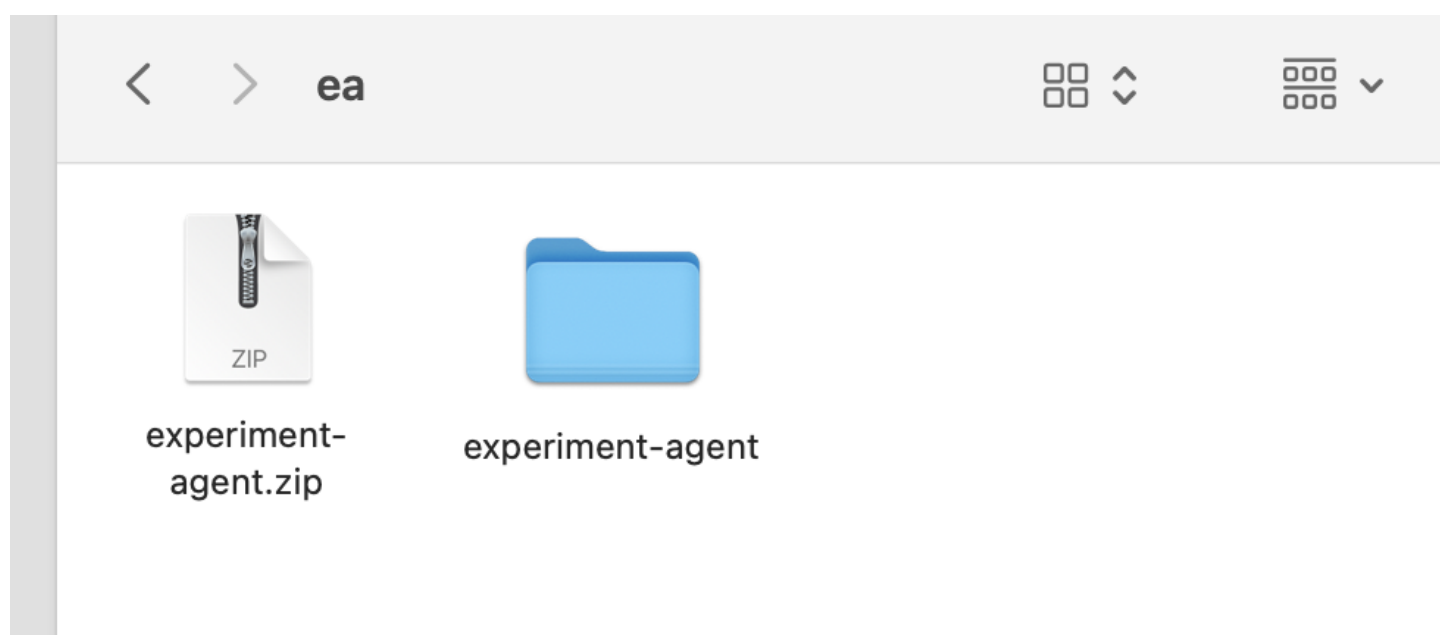
系统环境必须安装有conda与docker

源代码:

<https://github.com/hitsz-ids/AI-Course.git>

Agentic_AI_M2/ea复现/code/experiment-agent.zip

解压源代码到文件夹:

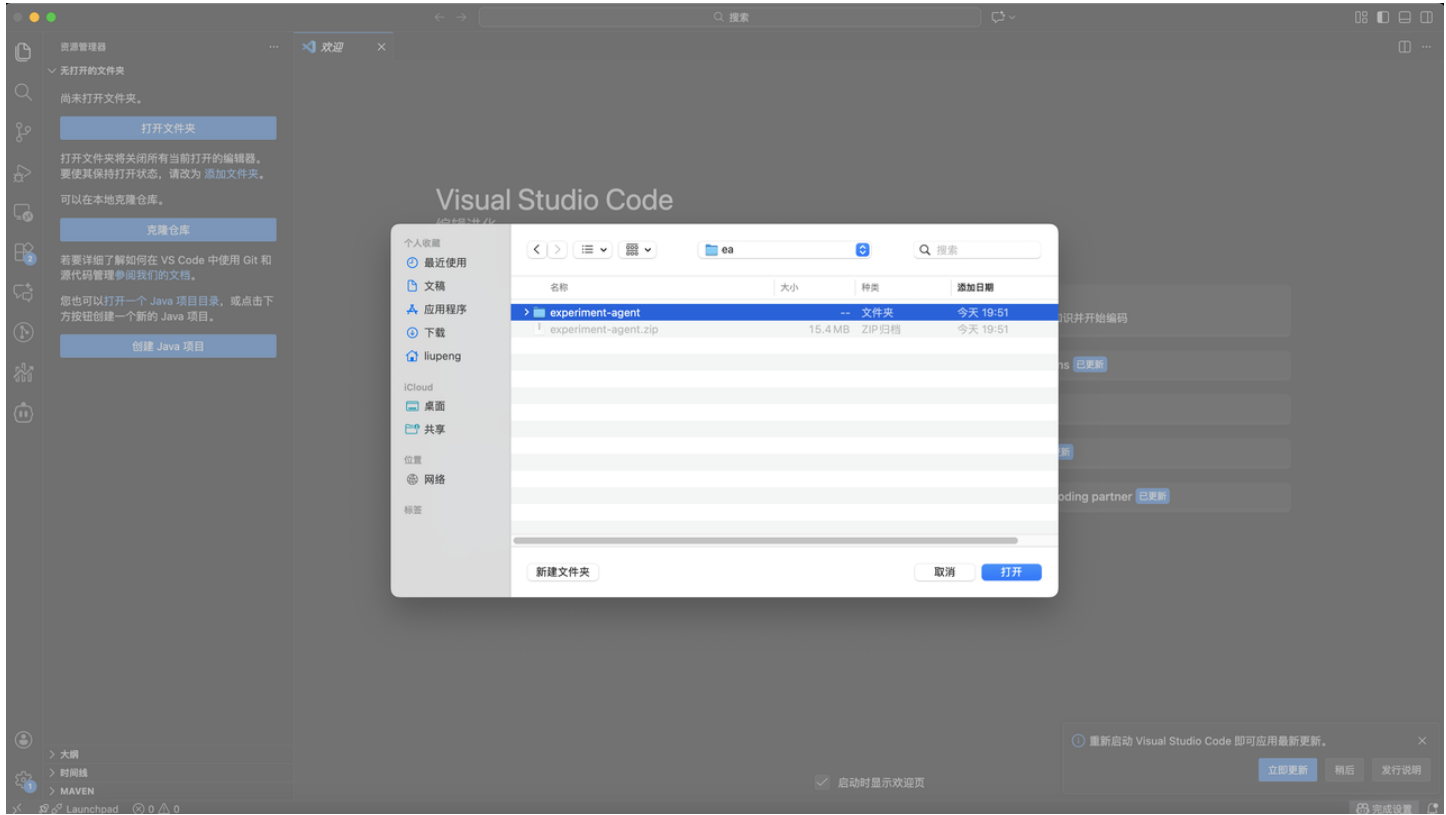


使用命令行解压:

代码块

- 1 #将压缩包放到文件夹解压
- 2 unzip experiment-agent.zip

打开VSCode选择已解压项目



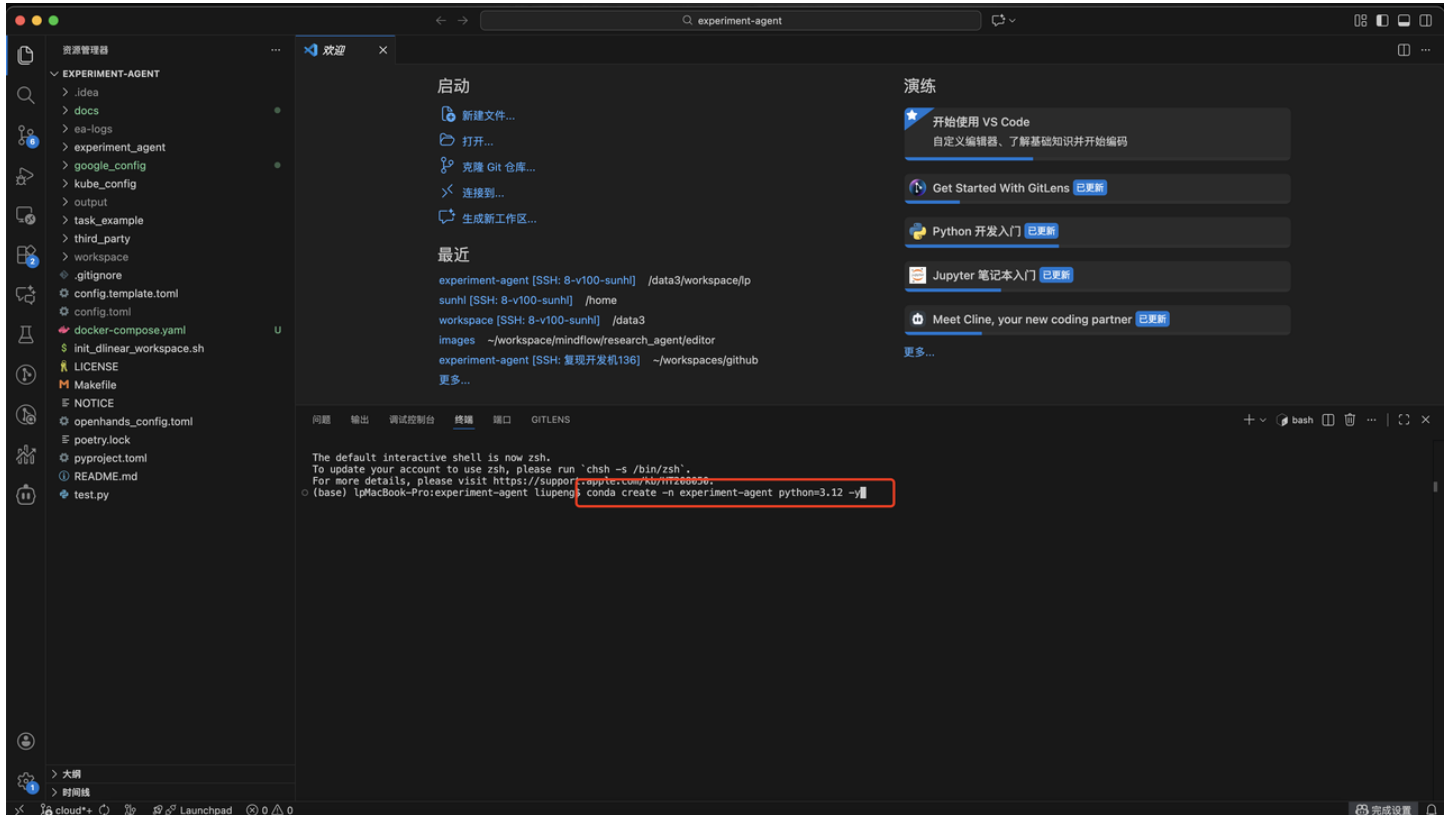
创建ea-python环境

1.新建conda环境

新建终端,并创建experiment-agent conda环境

代码块

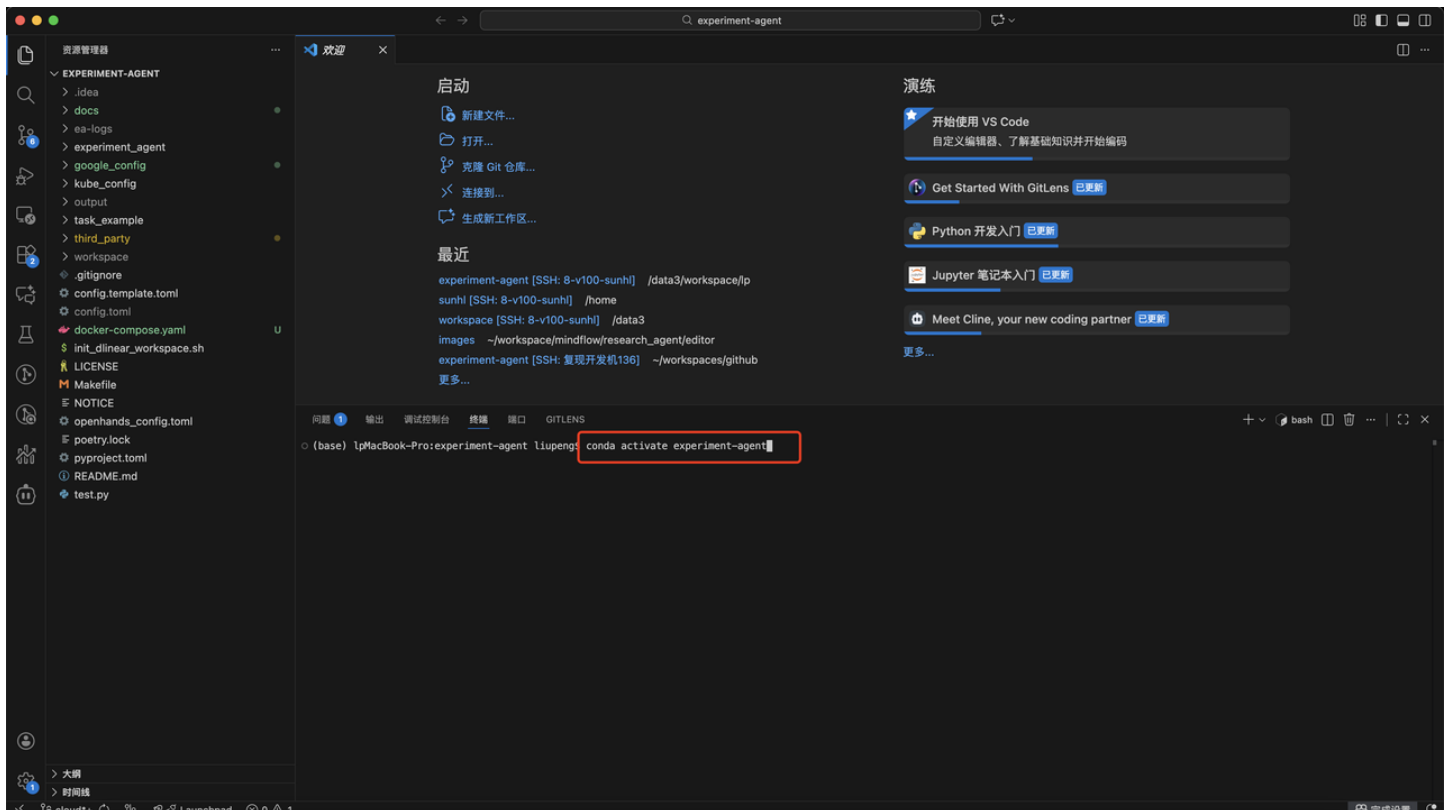
```
1      conda create -n experiment-agent python=3.12 -y
```



2. 激活conda环境

代码块

```
1 conda activate experiment-agent
```

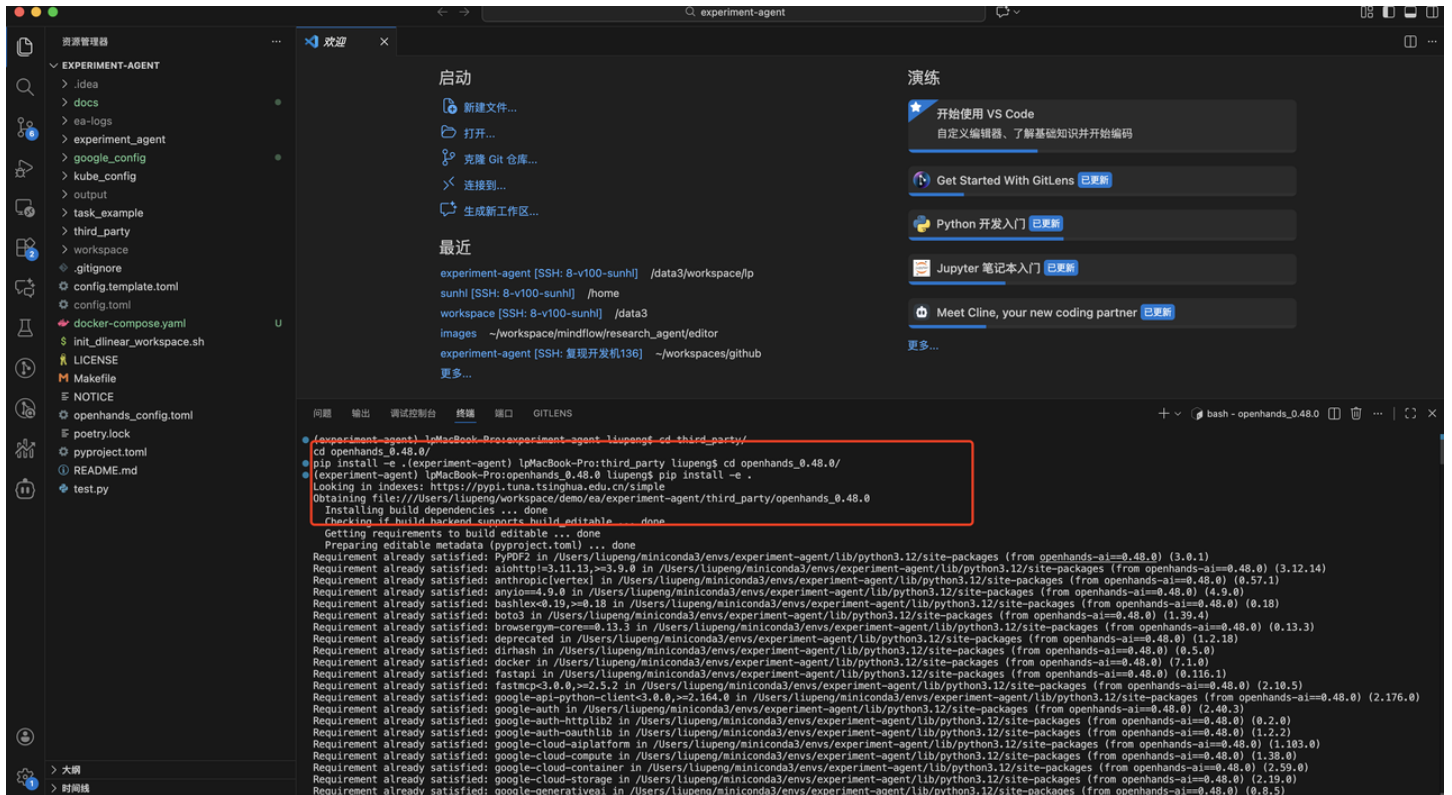


3. 依赖安装

3.1 openhands依赖安装

代码块

```
1 cd third_party/
2 cd openhands_0.48.0/
3 pip install -e .
```



完成依赖后终端指令退回到项目根目录

代码块

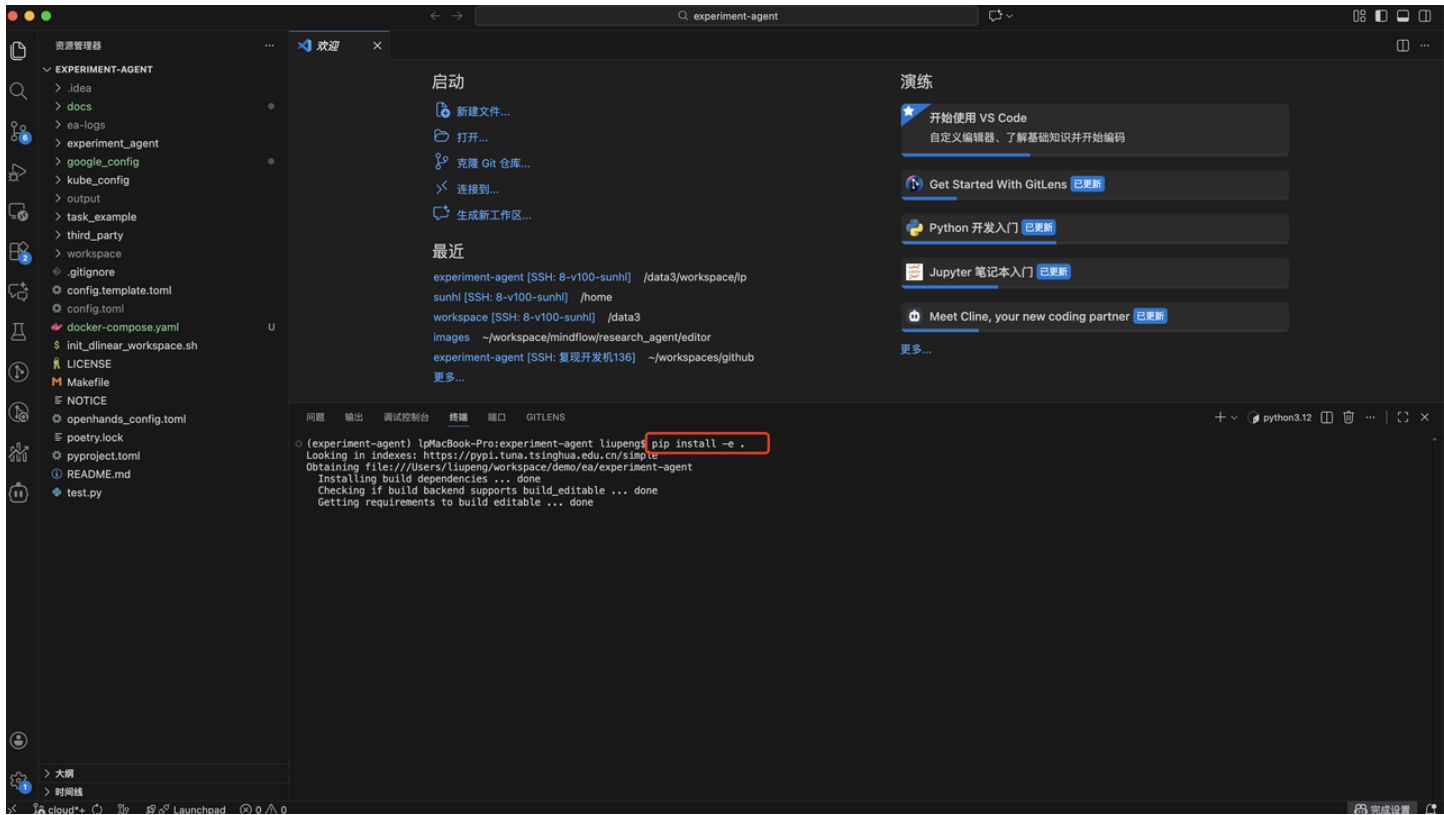
```
1 cd ../
2 cd ../
```

3.2 ea依赖安装

在ea项目根目录执行命令

代码块

```
1 pip install -e .
```



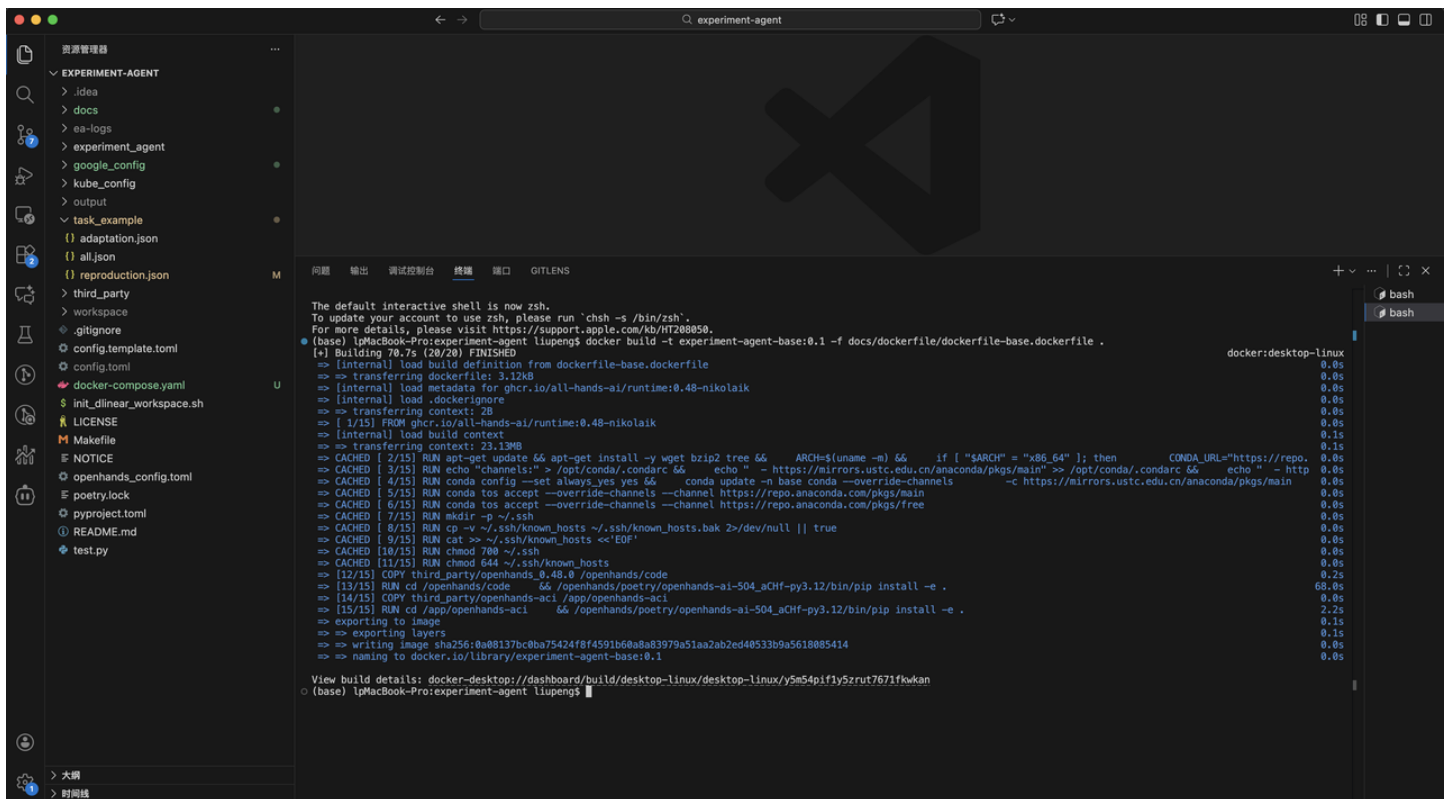
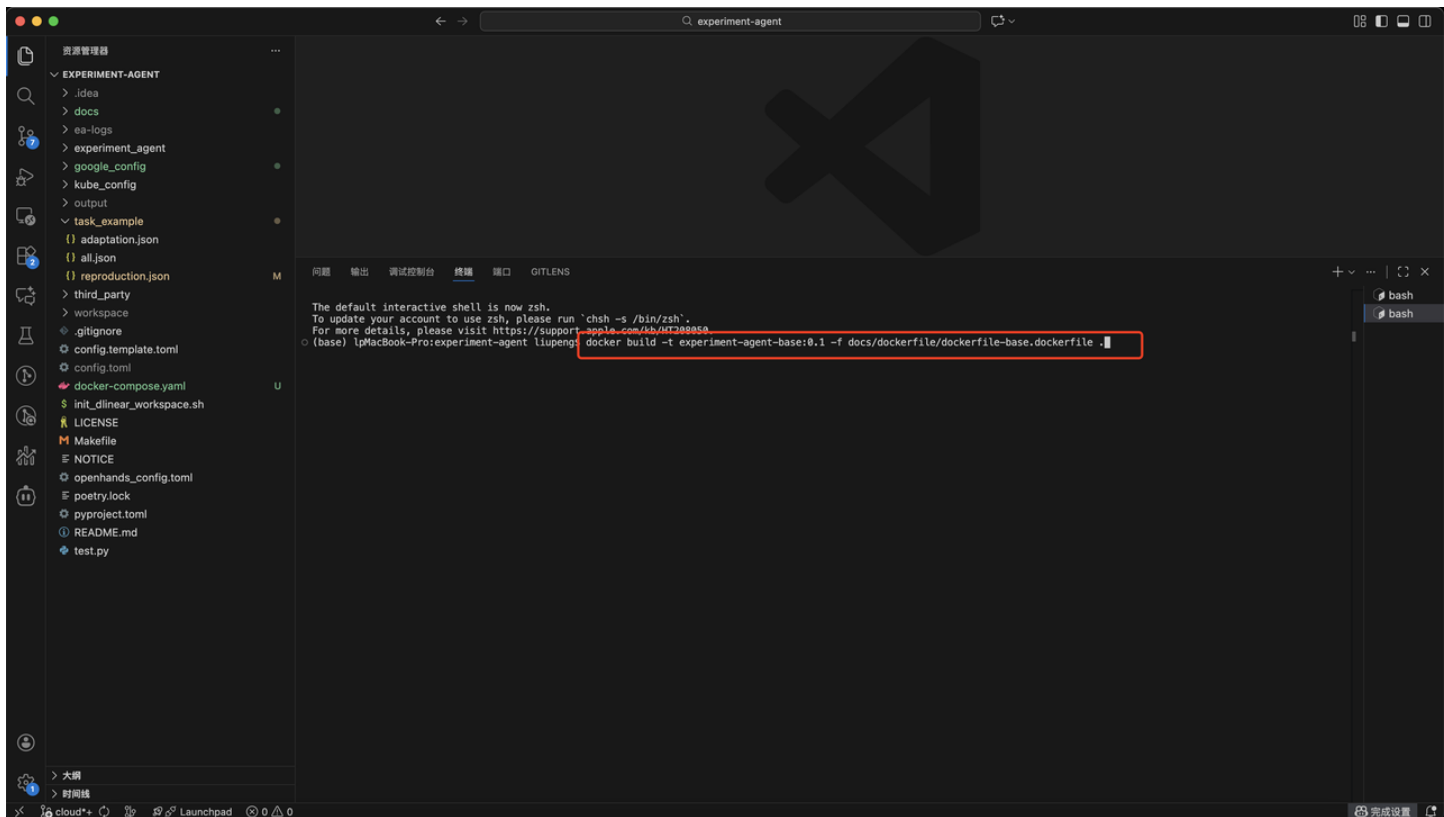
等待命令执行完毕,完成ea依赖安装

4.镜像构建

运行命令构建ea使用的基础镜像

代码块

```
1 docker build -t experiment-agent-base:0.1 -f docs/dockerfile/dockerfile-  
base.dockerfile .
```



复现TimeXer项目

检查配置

项目配置

检查项目根目录下config.toml文件配置

1.stage的值为reproduction

2.runtime的类型为docker

3.模型配置是否正确

```
config.toml
# agent执行的阶段操作, adaptation: 适配, reproduction: 复现, all: 复现+适配, 注意, 当前只支持reproduction
stage = 'reproduction'
# agent执行命令执行环境, docker 或者 google_k8s
runtime='docker'
# 是否开启gpu
enable_gpu = true
[google_k8s]
# gpu个数
gpu_count = 1

[local_docker]
# 对应项目、数据、评测程序的目录
workspace = './workspace'
# 配置对应的gpu的id如果有多个以,分割,例: cuda_visible_devices='1,2,3', 只有当runtime为docker并且在本地执行时生效
cuda_visible_devices = '7'

# 大模型接口配置
[llm]
# 模型名称, 例: claude-sonnet-4-20250514
#model = 'deepseek/deepseek-chat'
#api_key = 'sk-e99e84afee084326ab74cd762af95f48'
#base_url = 'https://api.deepseek.com'
model = "claude-sonnet-4-5-20250929"
api_key = "sk-1"
base_url = "https://poloai.top"
temperature = 0
# 发送给llm观察内容的最大字数
max_message_chars = 30_000

# 适配sota的相关配置
[adaptation]
# openhands的配置文件
openhands_config_path = './openhands_config.toml'
# 适配的运行的最大步数
max_iterations = 300
# agent的memory的存储类型, 目前有两种: file (文件)、memory (内存)
memory_storage_type = 'file'

# 复现sota的相关配置
[reproduction]
# openhands的配置文件
openhands_config_path = './openhands_config.toml'
# 适配的运行的最大步数
max_iterations = 300
# agent的memory的存储类型, 目前有两种: file (文件)、memory (内存)
memory_storage_type = 'file'
```

任务配置检查

查看task_example/reproduction.json的值

确认git_url是否为需要复现的项目地址:<https://github.com/thuml/TimeXer.git>

确认project_name是否为正确的项目名:TimeXer

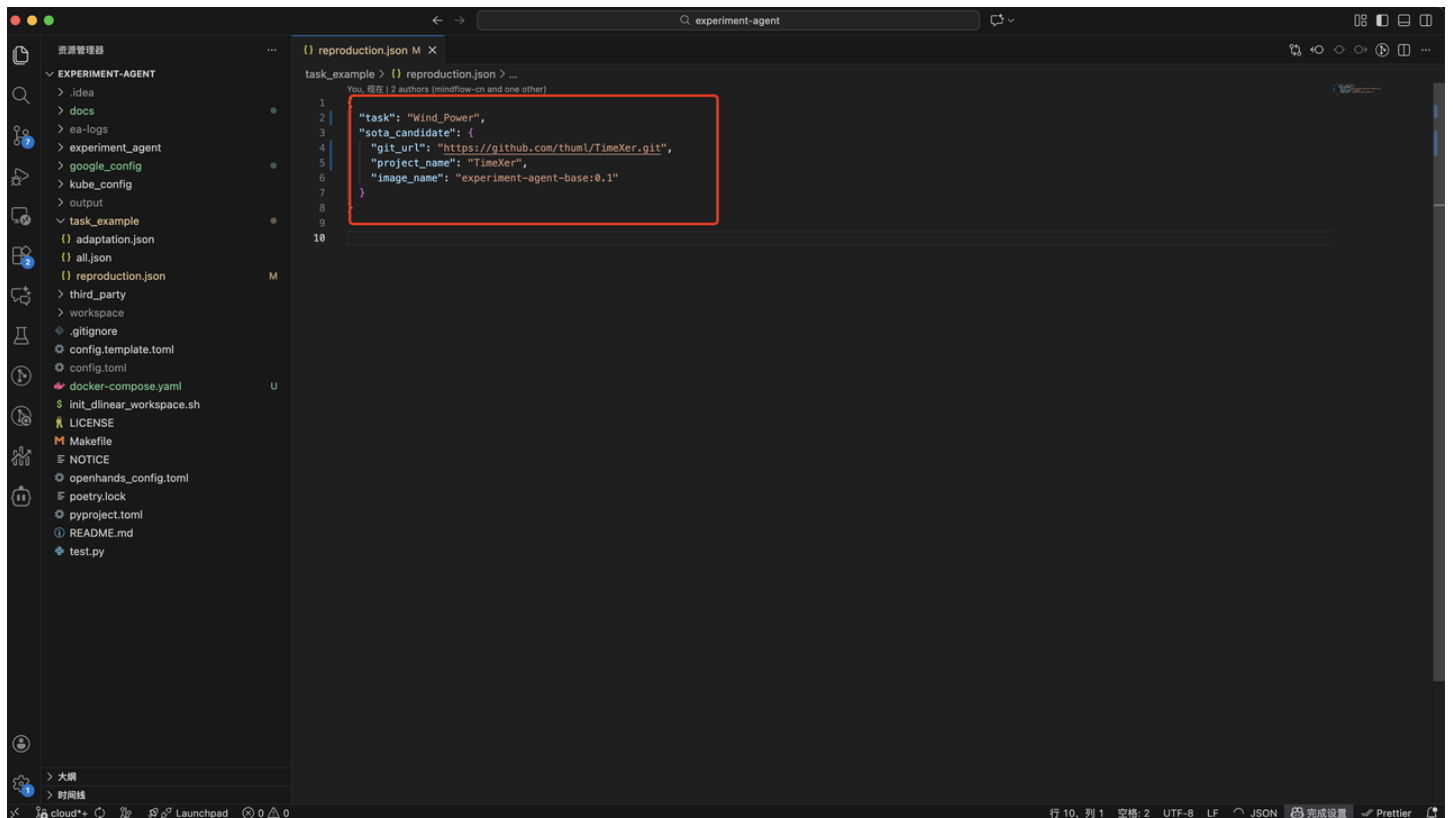
确认image_name是否为构建的ea基础镜像

代码块

```
1  {
2    "task": "Wind_Power",
3    "sota_candidate": {
4      "git_url": "https://github.com/thuml/TimeXer.git",
5      "project_name": "TimeXer",
6      "image_name": "experiment-agent-base:0.1"
7    }
}
```

```
8 }
```

```
9
```

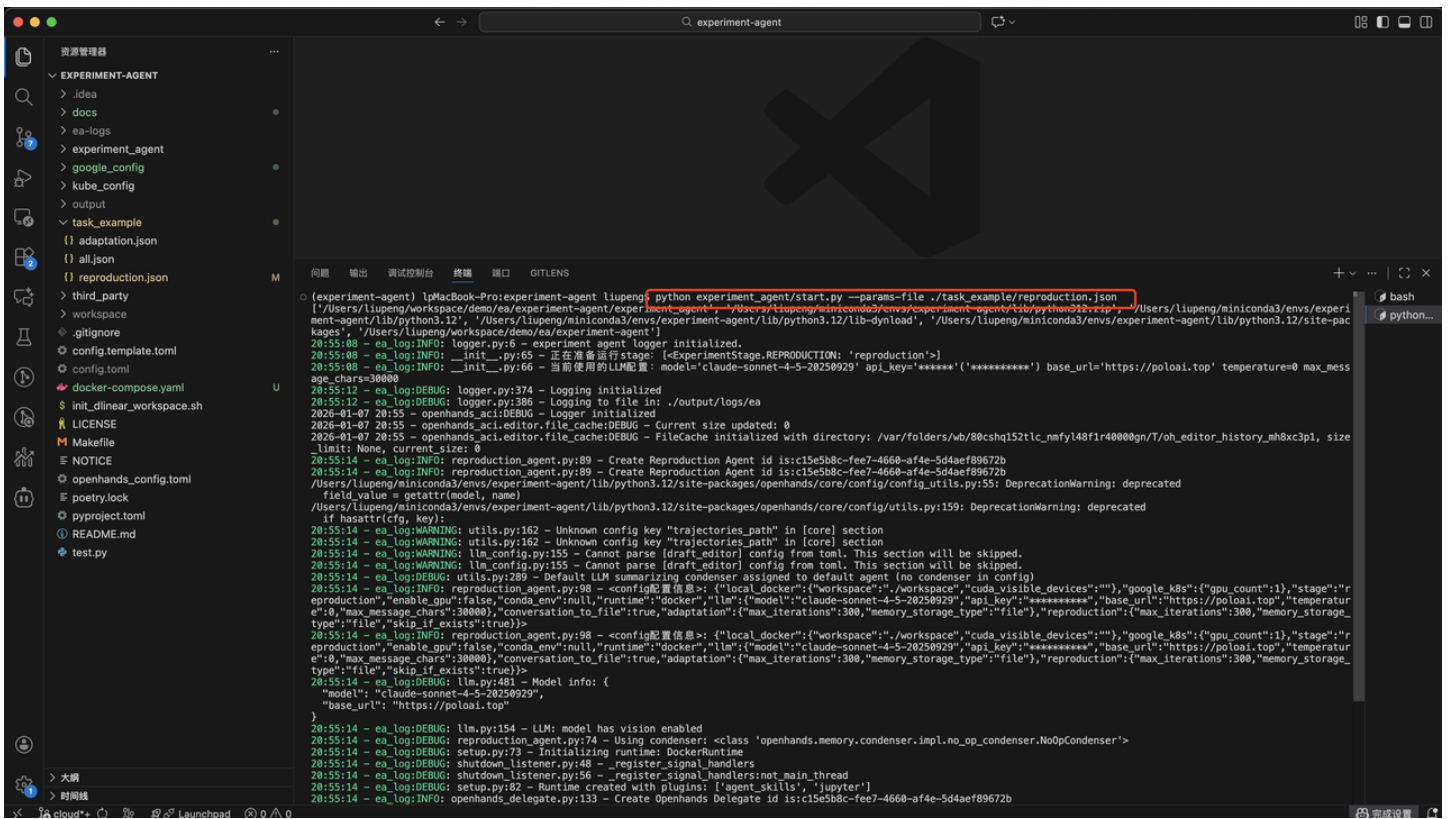


运行项目复现

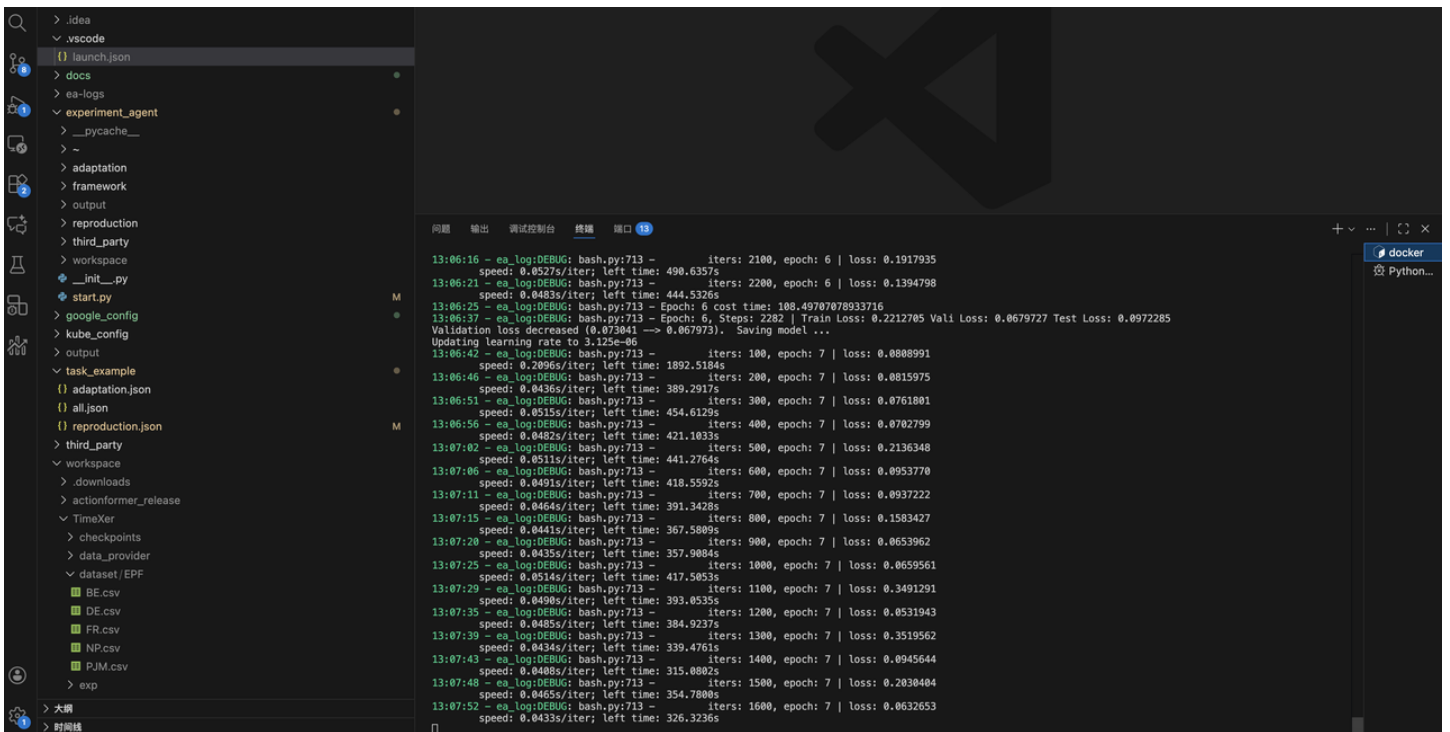
运行复现命令

代码块

```
1 python experiment_agent/start.py --params-file  
  ./task_example/reproduction.json
```



日志查看进入训练



复现DiGIT项目:

论文准备工作:

1.论文pdf转markdown文件:

1.1论文下载:

 Cornell University

We gratefully acknowledge support from the Simons Foundation, member institutions, and all contributors.
 [Donate](#)


 > cs > arXiv:2505.05711

Search... All fields Search

[Help](#) | [Advanced Search](#)

Computer Science > Computer Vision and Pattern Recognition

[Submitted on 9 May 2025]

DiGIT: Multi-Dilated Gated Encoder and Central-Adjacent Region Integrated Decoder for Temporal Action Detection Transformer

Ho-Joong Kim, Yearang Lee, Jung-Ho Hong, Seong-Whan Lee

In this paper, we examine a key limitation in query-based detectors for temporal action detection (TAD), which arises from their direct adaptation of originally designed architectures for object detection. Despite the effectiveness of the existing models, they struggle to fully address the unique challenges of TAD, such as the redundancy in multi-scale features and the limited ability to capture sufficient temporal context. To address these issues, we propose a multi-dilated gated encoder and central-adjacent region integrated decoder for temporal action detection transformer (DiGIT). Our approach replaces the existing encoder that consists of multi-scale deformable attention and feedforward network with our multi-dilated gated encoder. Our proposed encoder reduces the redundant information caused by multi-level features while maintaining the ability to capture fine-grained and long-range temporal information. Furthermore, we introduce a central-adjacent region integrated decoder that leverages a more comprehensive sampling strategy for deformable cross-attention to capture the essential information. Extensive experiments demonstrate that DiGIT achieves state-of-the-art performance on THUMOS14, ActivityNet v1.3, and HACS-Segment. Code is available at: [this https URL](#)

Comments: CVPR 2025
 Subjects: Computer Vision and Pattern Recognition (cs.CV)
 Cite as: arXiv:2505.05711 [cs.CV]
 (or arXiv:2505.05711v1 [cs.CV] for this version)
<https://doi.org/10.48550/arXiv.2505.05711>

Submission history

From: Ho-Joong Kim [\[view email\]](#)
 [v1] Fri, 9 May 2025 01:17:30 UTC (1,675 KB)

Bibliographic Tools
 Code, Data, Media
 Demos
 Related Papers
 About arXiv Labs

Bibliographic and Citation Tools

☐ Bibliographic Explorer (What is the Explorer?)

☐ Connected Papers (What is Connected Papers?)

Access Paper:

[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)
[view license](#)

Current browse context: cs.CV
 < prev | next >
 new | recent | 2025-05
 Change to browse by: cs

References & Citations

[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

Export BibTeX Citation

Bookmark

arXiv:2505.05711v1 [cs.CV] 9 May 2025

DiGIT: Multi-Dilated Gated Encoder and Central-Adjacent Region Integrated Decoder for Temporal Action Detection Transformer

Jung-Ho Kim Yearang Lee Jung-Ho Hong Seong-Whan Lee*

Dept. of Artificial Intelligence, Korea University, Seoul, Korea
hojoong.kim, yr.lee, jungho-hong, sw.lee@korea.ac.kr

Abstract

In this paper, we examine a key limitation in query-based detectors for temporal action detection (TAD), which arises from their direct adaptation of originally designed architectures for object detection. Despite the effectiveness of the existing models, they struggle to fully address the unique challenges of TAD, such as the redundancy in multi-scale features and the limited ability to capture sufficient temporal context. To address these issues, we propose a multi-dilated gated encoder and central-adjacent region integrated decoder for temporal action detection transformer (DiGIT). Our approach replaces the existing encoder that consists of multi-scale deformable attention and feedforward network with our multi-dilated gated encoder. Our proposed encoder reduces the redundant information caused by multi-level features while maintaining the ability to capture fine-grained and long-range temporal information. Furthermore, we introduce a central-adjacent region integrated decoder that leverages a more comprehensive sampling strategy for deformable cross-attention to capture the essential information. Extensive experiments demonstrate that DiGIT achieves state-of-the-art performance on THUMOS14, ActivityNet v1.3, and HACS-Segment. Code is available at: <https://github.com/Dotorii-HJ/DiGIT>

1. Introduction

Temporal action detection (TAD) is crucial for video understanding and supports a wide range of real-world applications, including video surveillance, summarization, and retrieval. TAD aims to detect action instances within untrimmed videos by identifying action classes along with their start and end times. The most existing TAD meth-

Figure 1. Convergence curves with InternVideo2 [39] features on THUMOS14 [13]. Our method boosts the previous query-based detectors like TE-TAD [14] and TaDR [26].

32, 34, 42, 45], and query-based [14, 26, 33, 35, 51].

Query-based detectors, inspired by DETR [3], have attracted interest because of their potential to eliminate reliance on hand-crafted components, such as the sliding window and non-maximum suppression (NMS). This capability derives from their adoption of a set-prediction mechanism, which enables an end-to-end detection process through a one-to-one matching paradigm. Among them, TE-TAD [14] enables a full end-to-end detection process by reformulating the coordinate representation based on recent DETR-based network architectures, such as multi-scale deformable attention [50]. However, despite these advancements, query-based detectors still rely on originally designed architectures for object detection, which is not fully suited to addressing the unique challenges of TAD. We identify two limitations within the encoder and decoder structures of existing query-based TAD models. (1) In the

<https://mineru.net/OpenSourceTools/Extractor>



点击上传文件,将pdf文件传入.



完成转换后,点击右上角下载markdown文件文件

4) 只写你能从“论文/代码/我提供的信息”中找到证据支撑的内容。任何推断必须标注【推断】并说明依据；找不到就写【未知】并明确缺失点。

5) 报告以“主实验场景”为最小单元组织。若主实验在多个数据集/任务上给出主表结果，则按“数据集/任务 = 场景”拆成多个独立实验（E1/E2...），每个都必须能单独复现，不依赖其他实验小节。

你将收到的输入

- PAPER: 论文标题、摘要、方法描述、实验章节、主结果表格/图、附录（可能是PDF抽取文本）

- REPO: 代码仓库信息（README、目录树、关键脚本、配置文件、requirements/environment、数据脚本、训练/评测脚本、Docker/conda等）

- EXTRA: 我额外提供的任何说明（比如数据路径、GPU信息等；没有就忽略）

你的工作流程（必须按顺序做；输出中体现结果，但不要写“我在思考”）

Step A. 主实验信息盘点与证据地图（先对齐，避免漏）

A1) 只列出论文的【主实验清单】（按数据集/任务场景划分），给每个主实验分配 Experiment ID（E1, E2...）。

- 同时给出“被排除实验清单（只列名称，不展开）”，用于证明你遵守范围约束。

A2) 列出代码仓库中与“主实验的数据准备/训练/评测/配置/复现论文主表结果”相关的关键入口文件清单（脚本、配置、Makefile、bash、python entry、notebook等）。

A3) 做“主实验映射表”：每个 Experiment ID 对应到论文证据（章节/表格/图）与代码证据（文件路径、函数/类/配置名、README命令）。

- 映射不到要标【未映射】并说明缺什么（例如缺评测脚本、缺数据预处理、缺配置、缺权重等）。

Step B. 输出正式报告（严格按下述格式）

报告必须包含：

0. 主实验复现结论总览（用于快速决策）

- 用一个总览表列出每个主实验：

Experiment ID	场景/数据集	任务	论文主指标与数值（引用表/图）	代码入口（命令/脚本/配置）	复现难度（低/中/高）	可复现性判断（可/部分可/不可）	主要风险点
---------------	--------	----	-----------------	----------------	-------------	------------------	-------

- “可复现性判断”必须有依据，不得拍脑袋。

1. 论文概述（只写与主实验理解/复现强相关的信息）

1.1 标题：【从 PAPER 提取】

1.2 方法一句话总结：讲清楚“输入是什么、输出是什么、核心机制是什么”

1.3 核心贡献（3-6条）：每条需能在论文中找到支撑；若是你归纳要标【归纳】

2. 主实验复现详解（最重要）

从 E1 开始，每个主实验用完全独立的小节，格式固定如下。注意：不要让 E2 依赖 E1 的描述，必要信息在本实验重复一遍。

【E? 主实验标题：.....（数据集/任务）】

A. 这个主实验在回答什么问题

- 实验目的/核心结论对应点

- 论文证据位置（章节/主结果表/图编号；若输入无编号，用“实验章节某段落/某表”）

B. 实验任务与工作原理（只保留与复现直接相关的）

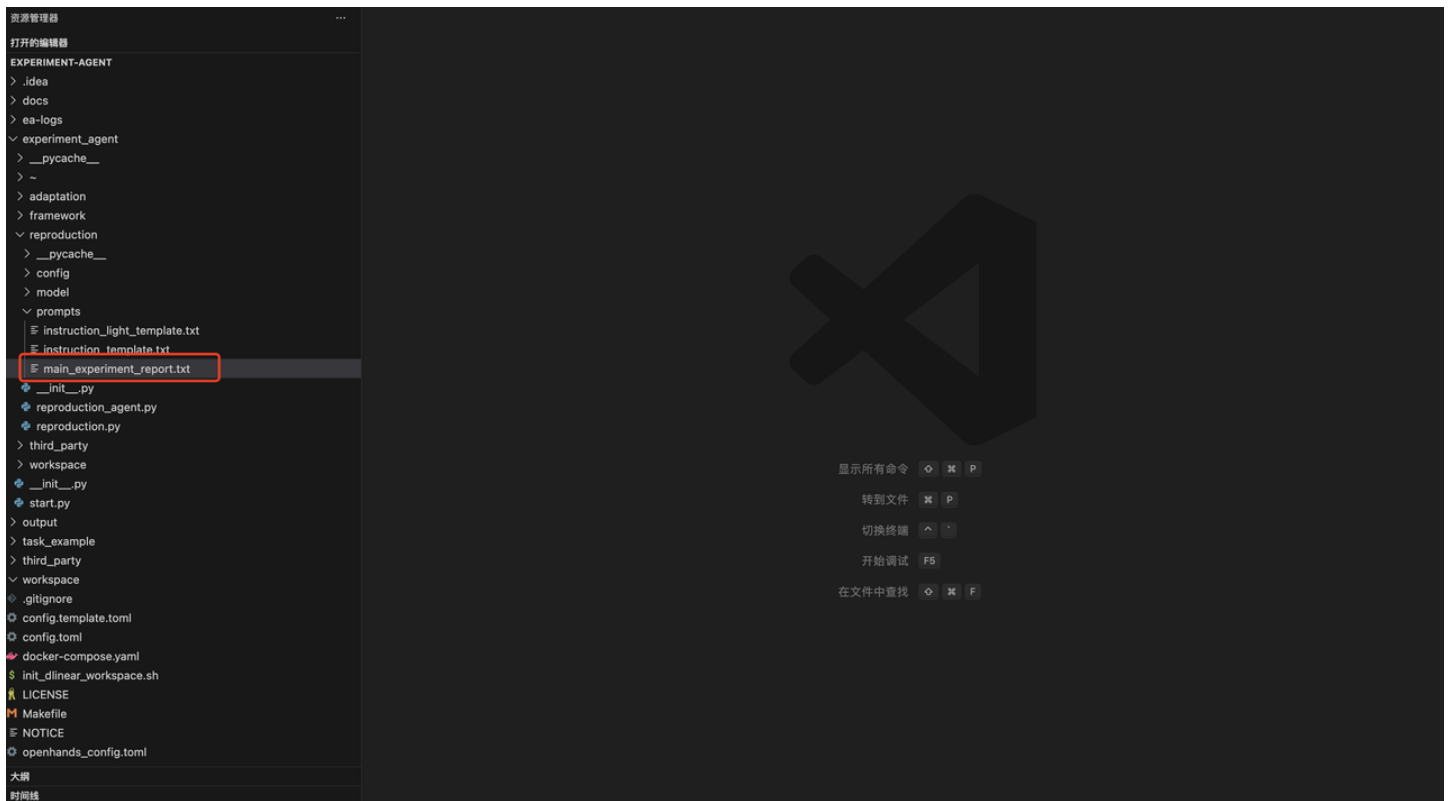
53 - 任务定义：输入/输出、预测目标、约束条件
54 - 方法在该实验下的关键流程：数据 → 模型 → 损失/解码 → 输出
55 - 说明该主实验的“最终设置”是什么（例如 full model、默认训练策略、官方评测协议）
56 - 举实例说明实验
57
58 C. 数据（必须足够让人真正拿到数据并喂给代码）
59 - 数据集名称与来源：从论文与代码提取（如有下载脚本/说明给出对应文件路径）
60 - 数据许可/访问限制（论文或README提到则写；没有则【未知】）
61 - 数据结构示例：用“字段名：类型/含义”的最小schema，或目录结构/样本结构；
62 如果代码里有 Dataset 类，按它的 `__getitem__` 返回内容说明
63 - 数据量：总量、train/val/test（或fold）大小；论文与代码不一致要指出矛盾，并说明可能原因
【推断】
64 - 训练集构建：过滤、采样、负采样、增强、tokenization、切分、label生成等；具体到脚本/配置
在哪里实现
65 - 测试集构建：同上
66 - 预处理与缓存：需要跑哪些脚本、会生成哪些文件、输出目录与命名规则
67
68 D. 模型与依赖
69 - 基础模型/Backbone：名称、版本、是否需下载权重、从哪里加载（代码路径/配置键）
70 - 关键模块：复现必须知道的模块与关键参数（能从代码推就写；推不了写【未知】）
71 - 训练策略：优化器、lr、scheduler、batch、梯度累积、混合精度、冻结策略、LoRA等
72 - 随机性控制：seed、deterministic设置、cudnn相关（从代码里找；没有则【未知】）
73
74 E. 评价指标与论文主表预期结果
75 - 指标定义：含义与计算方式（论文怎么说、代码怎么算；不一致必须指出）
76 - 论文主结果数值：逐项列出（可用小表格），并标注来源（主结果表/图/段落）
77 - 复现预期：优先使用论文主表值；如有多seed均值方差写出来；否则写“以论文主表数值为准”
78
79 F. 环境与硬件需求（复现必要信息，不做“效率实验”）
80 - 软件环境：Python/torch/cuda/transformers等版本（从
requirements/environment/README 提取）
81 - 硬件要求：论文或README提到的GPU型号/数量/显存/CPU/RAM/磁盘；没写则谨慎【推断】并说明
依据（如模型规模、batch、是否DDP）
82 - 训练时长：仅在论文/README/issue 有明确说明时引用；否则【未知】
83
84 G. 可直接照做的主实验复现步骤（按顺序给命令与产物）
85 把主实验写成“从零到出主表指标”的清单，每一步包含：
86 - 目的
87 - 命令（优先从README或脚本提取；若需要你自己拼命令，必须标【推断】并给出拼接依据：入口脚
本、argparse参数、配置键）
88 - 关键参数/配置文件路径
89 - 预期生成的文件/日志/指标输出位置
90 步骤至少包含：
91 1) 获取代码与安装依赖（conda/pip/docker任选其一或都给）
92 2) 获取数据与放置路径（含环境变量或配置项）
93 3) 预处理（若需要）
94 4) 训练（含resume/checkpoint、保存策略）

```

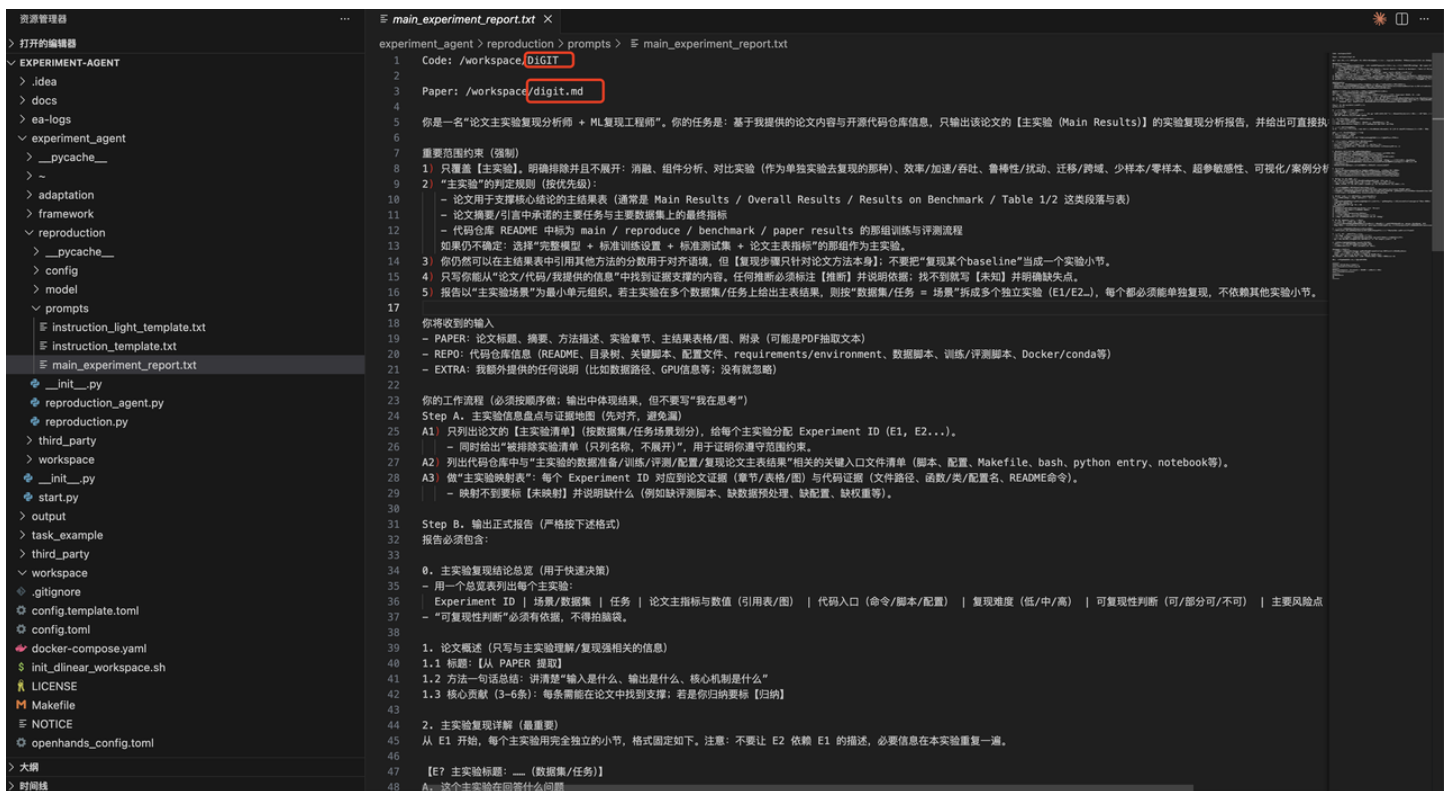
95 5) 主实验评测/推理（加载哪个checkpoint、输出格式）
96 6) 主表指标对齐：如何从日志/输出文件得到与论文主表一致的指标
97
98 H. 可复现性判断（必须给结论 + 证据）
99 - 结论：可复现 / 部分可复现 / 不可复现
100 - 依据清单：数据不可得、权重不可得、缺少关键脚本/配置、版本锁死、私有预处理、评测协议不
    明、论文与代码不一致等
101 - 补救路径（基于仓库内容；若是通用经验要标【经验】）：例如只跑评测、使用作者checkpoint、
    改用可得的数据版本、从issue定位commit等
102
103 I. 主实验专属排错要点（只写与该仓库强相关的坑）
104 - 路径约定、必须存在的文件、tokenizer版本、数据编码、分布式启动参数、fp16溢出、评测脚本
    字段名等
105
106 3. 主实验一致性检查（只围绕主实验）
107 - 论文主表指标是否可能被仓库脚本直接产出同款结果
108 - 多个主实验是否共享同一套预处理与评测入口（指出共用入口，但不让实验章节互相依赖）
109 - 最小复现路径：如果只想最快验证主表分数，建议按什么顺序跑哪些命令，为什么
110
111 4. 未知项与我需要你补充的最小信息（尽量少问）
112 - 只有当主实验复现关键步骤无法确定时才列出问题
113 - 每个问题都要解释：为什么必须、缺了会导致什么后果
114
115 证据指向格式（强制）
116 - 每当你给出关键信息（命令、指标、数据量、版本、入口脚本、路径约定），在句末附上证据：
117   [Paper: 主实验章节/主结果表X/图Y/段落描述]
118   [Repo: path/to/file.py 的函数/类/参数名 或 README 某段]
119 如果输入不含行号，用“文件路径 + 关键标识符（函数名/参数名/配置键）”定位。
120
121 现在开始分析。不要输出与任务无关的内容。
122
123 =====
124 PAPER（论文内容/摘录/抽取文本）
125 请查看根目录下llm-attacks/gcg.md
126 =====
127 REPO（仓库信息：建议包含目录树 + README + 关键脚本/配置）
128 请查看根目录下llm-attacks/
129 =====
130 EXTRA（可选）
131 无
132 =====

```

将prompt放到experiment_agent/reproduction/prompts/目录下命名main_experiment_report.txt



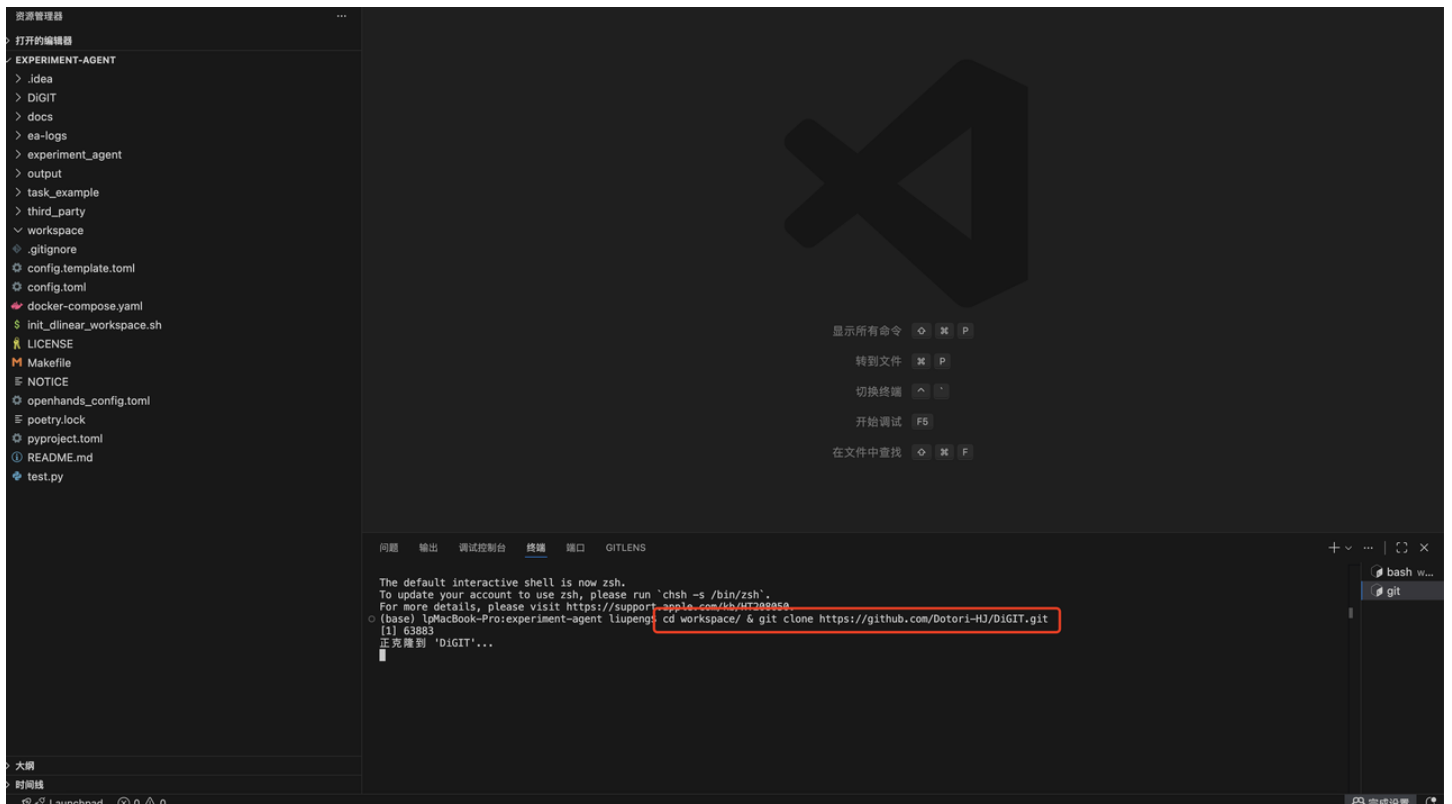
2.2修改主实验报告prompt为DiGIT项目



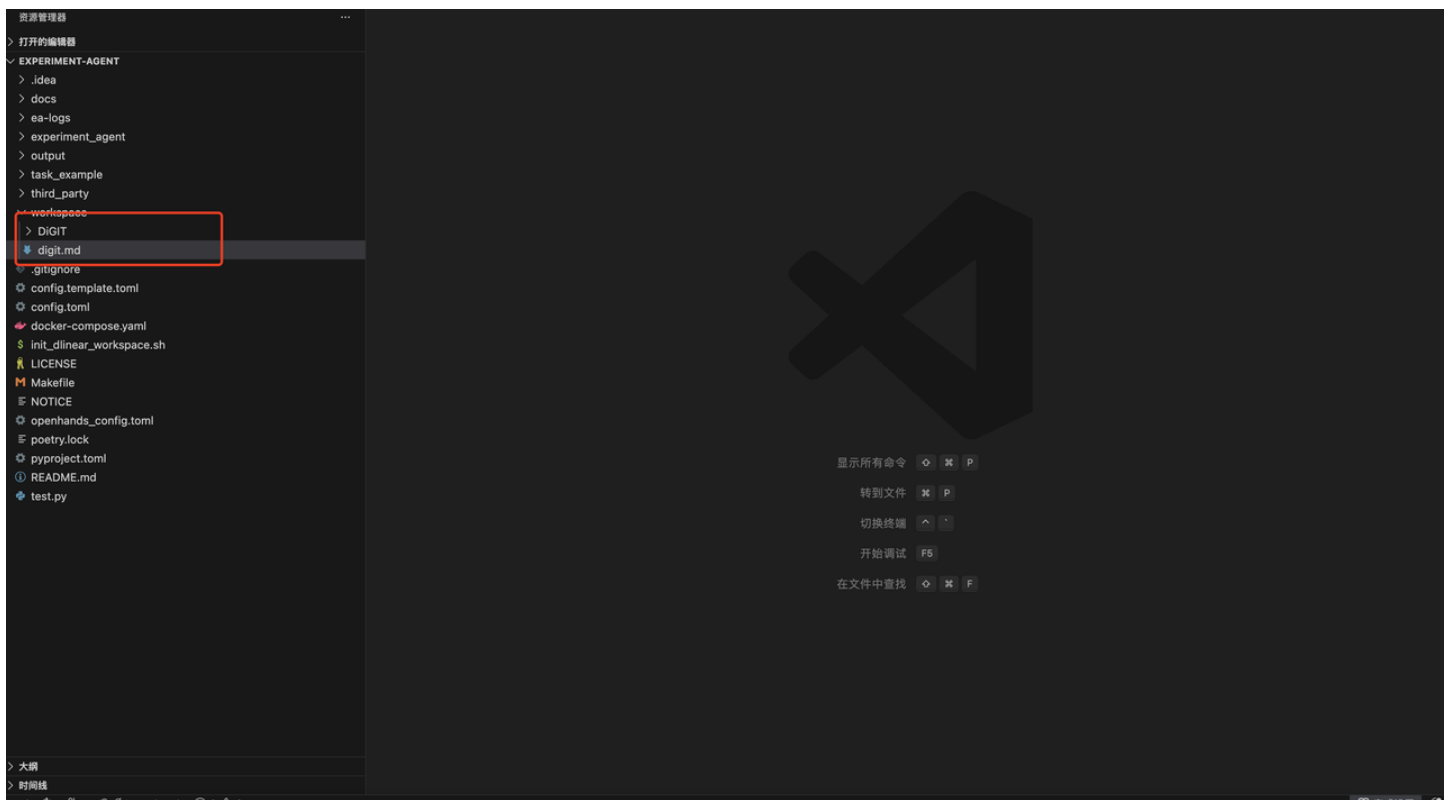
2.3 下载相关工作代码到workspace

代码块

```
1 cd ./workspace
2 git clone https://github.com/Dotori-HJ/DiGIT.git
```

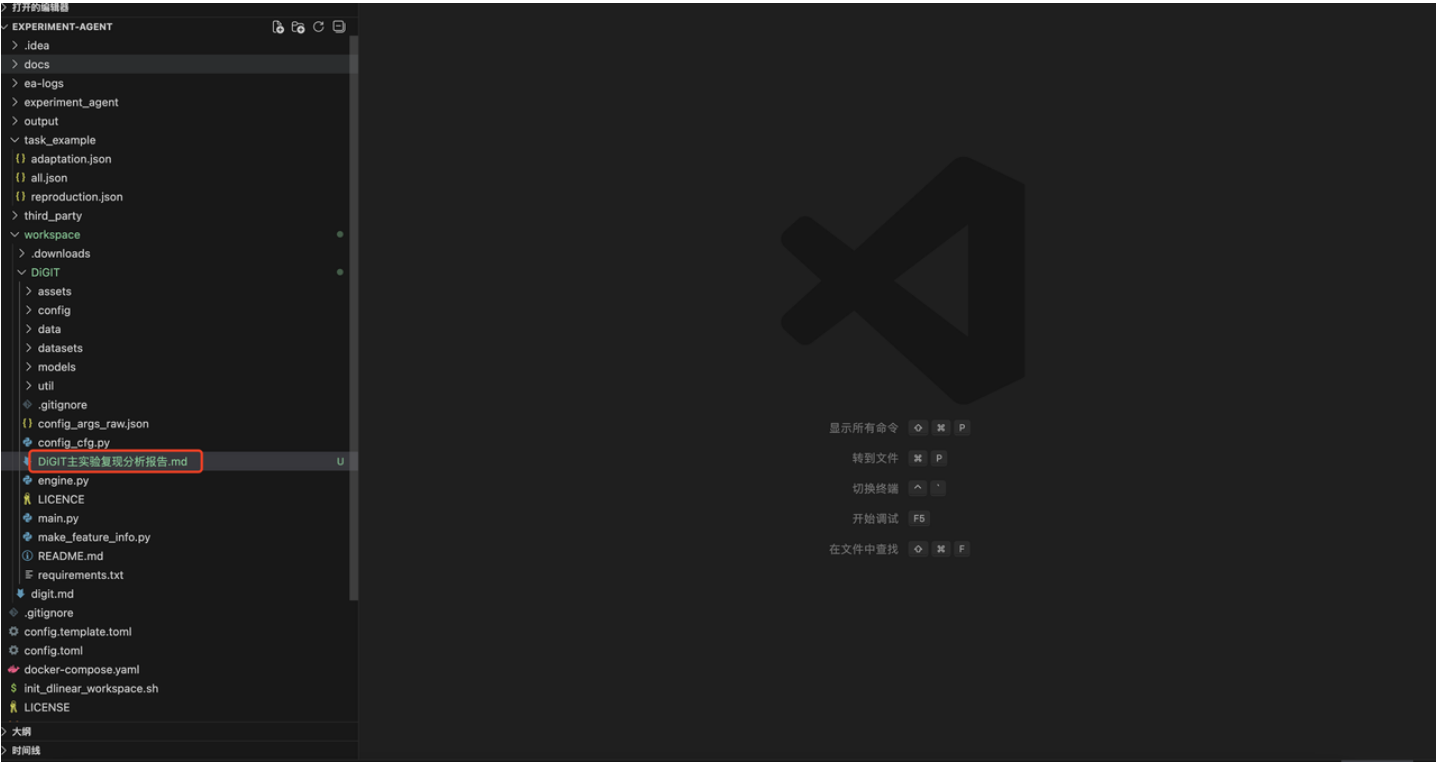


2.4 将论文markdown放入workspace文件夹



2.5 运行ea生成主实验报告

DiGIT项目下会生成一个markdown文件



MAIN_EXPERIMENT_REPRODUCTION_REPORT.md

DiGIT 主实验复现分析报告

Step A. 主实验信息盘点与证据地图

A1) 主实验清单 (Experiment ID)

根据论文Table 1 (THUMOS14和ActivityNet v1.3) 和Table 2 (HACS-Segment) , 论文的主实验针对时序动作检测 (Temporal Action Detection, TAD) 任务, 在三个标准benchmark数据集上验证DiGIT的性能。

Experiment ID	数据集	任务	论文证据位置
E1	THUMOS14	Temporal Action Detection (mAP@0.3-0.7)	Table 1, Section 4.2
E2	ActivityNet v1.3	Temporal Action Detection (mAP@0.5/0.75/0.95/Avg)	Table 1, Section 4.2
E3	HACS-Segment	Temporal Action Detection (mAP@0.5/0.75/0.95/Avg)	Table 2, Section 4.2

被排除实验清单 (范围约束) :

- 消融实验: MDGE超参 (Figure 5) 、CAID decoder序列分析 (Table 3) 、组件贡献分析 (Table 4) 、query selection分析 (Table 5)
- 收敛曲线分析 (Figure 1, Figure 6)
- CKA相似度可视化 (Figure 2)
- 采样点可视化 (Figure 8)
- 编码器输出特征余弦相似度可视化 (Figure 7)
- TadTR+Ours对比实验 (作为baseline增强的变体, 非论文主方法DiGIT本身的主实验)

A2) 代码仓库关键入口文件清单

文件/目录	用途
main.py	训练与评测的主入口脚本 [Repo: main.py]
config/digit/internvideo2/thumos14.py	THUMOS14主实验配置 [Repo: config/digit/internvideo2/thumos14.py]
config/digit/internvideo2/anet.py	ActivityNet v1.3主实验配置 [Repo: config/digit/internvideo2/anet.py]

3.查看主实验报告并分析

DiGIT 主实验复现分析报告

Step A. 主实验信息盘点与证据地图

A1) 主实验清单（Experiment ID）

根据论文Table 1（THUMOS14和ActivityNet v1.3）和Table 2（HACS-Segment），论文的主实验针对时序动作检测（Temporal Action Detection, TAD）任务，在三个标准benchmark数据集上验证DiGIT的性能。

Experiment ID	数据集	任务	论文证据位置
E1	THUMOS14	Temporal Action Detection (mAP@0.3-0.7)	Table 1, Section 4.2
E2	ActivityNet v1.3	Temporal Action Detection (mAP@0.5/0.75/0.95/Avg)	Table 1, Section 4.2
E3	HACS-Segment	Temporal Action Detection (mAP@0.5/0.75/0.95/Avg)	Table 2, Section 4.2

被排除实验清单（范围约束）：

- 消融实验：MDGE超参（Figure 5）、CAID decoder序列分析（Table 3）、组件贡献分析（Table 4）、query selection分析（Table 5）
- 收敛曲线分析（Figure 1, Figure 6）
- CKA相似度可视化（Figure 2）
- 采样点可视化（Figure 8）
- 编码器输出特征余弦相似度可视化（Figure 7）
- TadTR+Ours对比实验（作为baseline增强的变体，非论文主方法DiGIT本身的主实验）

0. 主实验复现结论总览

Experiment ID	场景/数据集	任务	论文主指标与数值	代码入口	复现难度	可复现性判断	主要风险点
1	THUMOS14	TAD	mAP@0.3=87.6, @0.5=77.6, @0.7=52.5, Avg=73.8 (NMS) [Paper: Table 1]	<code>python main.py --c config/digit/internvideo2/thumos14.py</code>	中	部分可	InternVideo2 特征需外部获取；无预训练权重提供
2	ActivityNet v1.3	TAD	mAP@0.5=62.0, @0.75=43.1, @0.95=11.3, Avg=42.0 (NMS) [Paper: Table 1]	<code>python main.py --c config/digit/internvideo2/onet.py</code>	中-高	部分可	特征+GT+外部分类分数均需外部获取
3	HACS-Segment	TAD	mAP@0.5=62.4, @0.75=47.9, @0.95=17.6, Avg=45.9 (NMS) [Paper: Table 2]	<code>python main.py --c config/digit/internvideo2/hacs.py</code>	中-高	部分可	特征+GT+外部分类分数均需外部获取

总结:

通过查看主实验报告,我们可以看到,论文中实验是针对于时间动作定位任务DiGIT在3个标准数据集上的表现.每个实验都契合我们的SOW任务,那么我们就可以选择E1作为我们复现的切口实验.

E1实验:

【E1 主实验标题：THUMOS14数据集上的时序动作检测】

A. 实验目的

- 实验目的：验证DiGIT在THUMOS14数据集上的时序动作检测性能，与现有方法（包括anchor-based、anchor-free、query-based）进行比较
- 核心结论：DiGIT在使用InternVideo2特征时取得mAP@Avg=73.8（带NMS），显著优于TE-TAD（70.3）和TadTR（67.3）等baseline [Paper: Table 1, Section 4.2]
- 论文证据位置：Table 1（THUMOS14列），Section 4.2 "THUMOS14"段落

B. 实验任务与工作原理

任务定义：

- 输入：视频的预提取特征序列 $X \in \mathbb{R}^{C \times T}$ （ $C=3200$ 为InternVideo2特征维度， T 为时序长度）
- 输出：检测到的动作实例集合 $\{(c_i, s_i, e_i, p_i)\}$ ，其中 c_i 为类别， s_i, e_i 为起止时间， p_i 为置信度
- 预测目标：最小化bipartite matching loss（分类focal loss + 回归Giou loss + log-ratio distance loss）[Paper: Section 3.3, Eq.12]

方法关键流程：

- 特征嵌入：输入特征通过ConvBackbone投影到hidden_dim=512维
- MDGE编码：6层Multi-Dilated Gated Encoder，每层使用多膨胀卷积（kernel_size=3, gc_kernel_size=11）捕获不同感受野的时序特征
- Multi-scale Adapter：将单尺度编码器输出转换为多尺度特征供decoder和query selection使用
- CAID解码：6层decoder，每层依次执行self-attention → central-region cross-attention → adjacent-region cross-attention → FFN
- 输出头：MLP预测类别logits和segment（center, width）

最终设置：

- 使用InternVideo2 6B特征（window=16, stride=4）
- 完整DiGIT模型（MDGE + CAID）
- 带NMS后处理（nms_thr=0.75）
- 测试集：THUMOS14 test split（213视频）

实例说明：

数据集准备:

[Repo: nms.py]

预期产物：编译后的NMS和deformable attention库

步骤2：获取数据与放置路径

2.1 获取THUMOS14 Ground Truth

```
# 仓库已包含，确认文件存在
ls data/thumos14/thumos14.py # 实际为JSON格式的GT
ls data/thumos14/th14_i3d2s_act_ft_info.json
```

[Repo: data/thumos14/]

2.2 获取InternVideo2特征

```
# 参考Video Mamba Suite获取特征
# https://github.com/OpenGVLab/video-mamba-suite/blob/main/video-mamba-suite/temporal-action-localization/README.md

# 下载后放置于指定路径
mkdir -p data/features/THUMOS14/thumos14_internvideo2_6b_w16_s4/thumos14_6b

# 特征文件命名格式：{video_name}_spatial_feature.pt
# 例如：video_test_0000004_spatial_feature.pt
```

[Repo: config/digit/internvideo2/thumos14.py, feature_folder]

【推断】：根据config中的 `name_format='{ }_spatial_feature.pt'`，特征文件应为PyTorch tensor格式。

步骤3：预处理（若需要）

THUMOS14已提供feature info，无需额外预处理。若使用其他数据集：

```
# 修改make_feature_info.py中的root和gt_path
python make_feature_info.py
```

[Repo: make_feature_info.py]

https://huggingface.co/datasets/cg1177/thumos14_internvideo2_6b_w16_s4

检查配置

项目配置：

检查项目根目录下config.toml文件配置

1.stage的值为reproduction

2.runtime的类型为docker

3.模型配置是否正确

```

1 # agent执行的阶段操作, adaptation: 适配, reproduction: 复现, all: 复现+适配, 注意, 当前只支持reproduction
2 stage = 'reproduction'
3 # agent执行命令执行环境, docker 或者 google_k8s
4 runtime='docker'
5 # 是否开启gpu
6 enable_gpu = true
7 [google_k8s]
8 # gpu个数
9 gpu_count = 1
10
11 [local_docker]
12 # 对应项目、数据、评测程序的目录
13 workspace = './workspace'
14 # 配置对应的gpu的id如果有多个以,分割, 例: cuda_visible_devices='1,2,3', 只有当runtime为docker并且在本地执行时生效
15 cuda_visible_devices = '7'
16
17 # 大模型接口配置
18 [llm]
19 # 模型名称, 例: claude-sonnet-4-20250514
20 #model = 'deepseek/deepseek-chat'
21 #api_key = 'sk-e99e84afee084326ab74cd762af95f48'
22 #base_url = 'https://api.deepseek.com'
23 model = "claude-sonnet-4-5-20250929"
24 api_key = " "
25 base_url = "https://poloai.top"
26 temperature = 0
27 # 发送给llm观察内容的最大字符数
28 max_message_chars = 30_000
29
30 # 适配sota的相关配置
31 [adaptation]
32 # openhands的配置文件
33 openhands_config_path = './openhands_config.toml'
34 # 适配的运行的最大步数
35 max_iterations = 300
36 # agent的memory的存储类型, 目前有两种: file (文件)、memory (内存)
37 memory_storage_type = 'file'
38
39 # 复现sota的相关配置
40 [reproduction]
41 # openhands的配置文件
42 openhands_config_path = './openhands_config.toml'
43 # 适配的运行的最大步数
44 max_iterations = 300
45 # agent的memory的存储类型, 目前有两种: file (文件)、memory (内存)
46 memory_storage_type = 'file'
47
48

```

任务配置检查

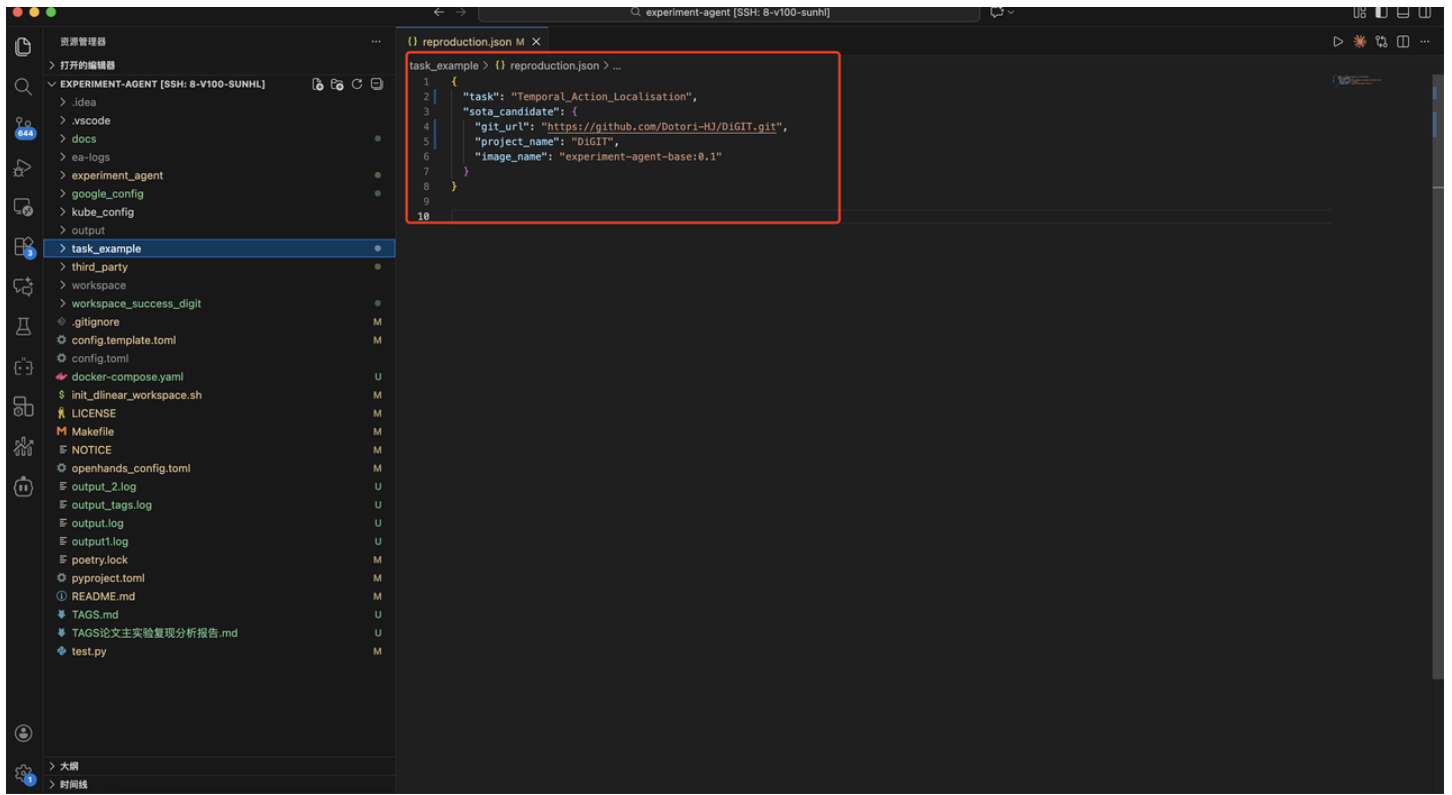
查看task_example/reproduction.json的值

代码块

```

1  {
2      "task": "Temporal_Action_Localisation",
3      "sota_candidate": {
4          "git_url": "https://github.com/Dotori-HJ/DiGIT.git",
5          "project_name": "DiGIT",
6          "image_name": "experiment-agent-base:0.1"
7      }
8  }
9

```



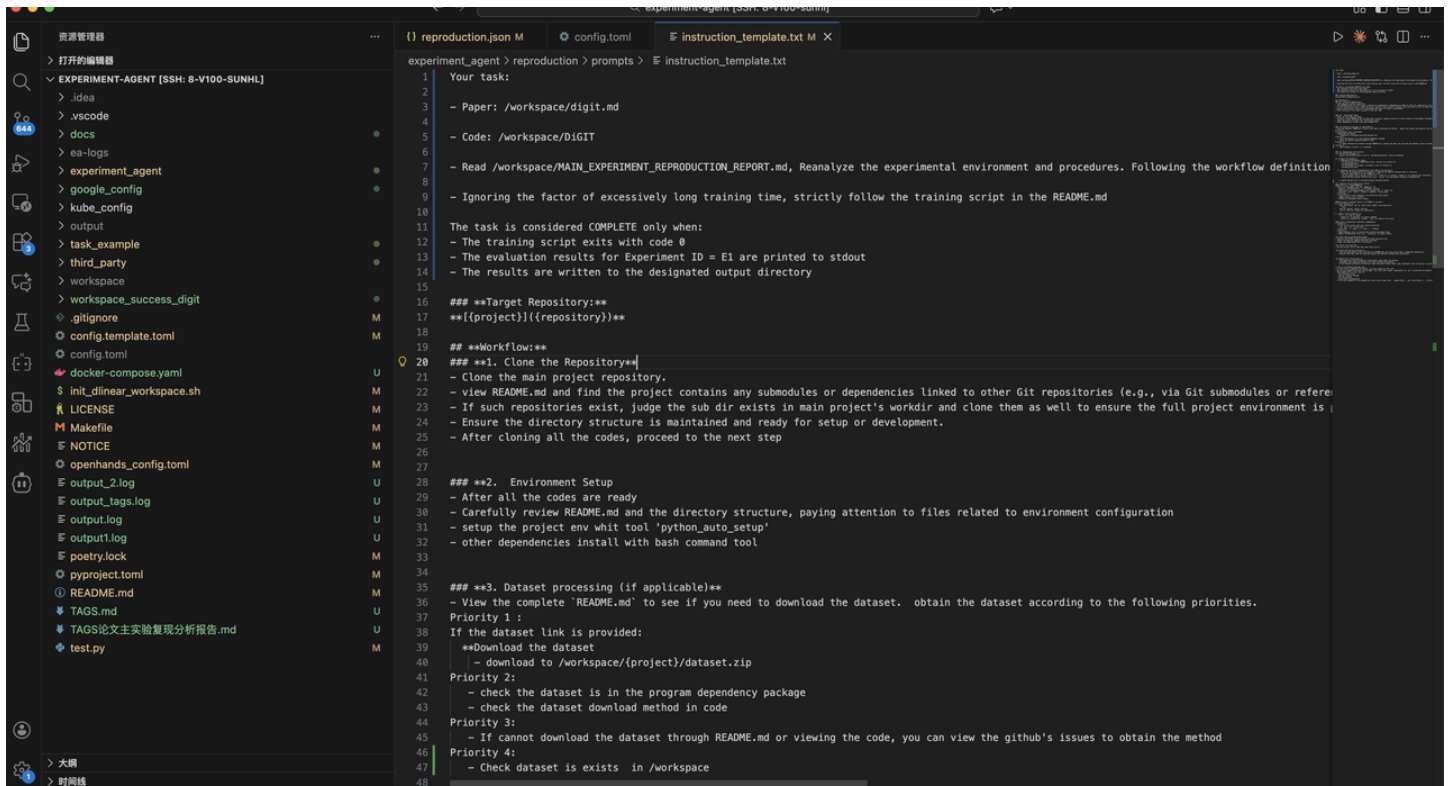
复现prompt修改:

为了防止agent认为任务训练过长,提前结束任务,需要再prompt开头中增加结束提示:

结合我们选定的E1实验

代码块

- ```
1 Your task:
2
3 - Paper: /workspace/{paper_markdown}
4
5 - Code: /workspace/{project}
6
7 - Read /workspace/MAIN_EXPERIMENT_REPRODUCTION_REPORT.md, Reanalyze the
 experimental environment and procedures. Following the workflow definition
 under, reproduce the experiment with Experiment ID = {experimentId}, complete
 the entire training and evaluation process, provide the final experimental
 evaluation results.
8
9 - Ignoring the factor of excessively long training time, strictly follow the
 training script in the README.md
10
11 The task is considered COMPLETE only when:
12 - The training script exits with code 0
13 - The evaluation results for Experiment ID = {experimentId} are printed to
 stdout
14 - The results are written to the designated output directory
```



## 运行项目复现:

### 代码块

```
1 python experiment_agent/start.py --params-file
 ./task_example/reproduction.json
```